ALA-TOO INTERNATIONAL UNIVERSITY

ALGORITHMIZATION AND PROGRAMMING

PART II

Spring Semester 2025-2026

***

Premature optimization is the root of all evil.

— Donald Knuth

WORLD TWO

STRUCTURED ALGORITHMS

REDUCING SEARCH THROUGH STRUCTURE

Algorithms that exploit known properties

Avoid unnecessary computation

Foundation of scalable systems

WHY WORLD TWO MATTERS

BRUTE FORCE FAILS AT SCALE

Exponential growth is unsustainable

Real systems handle millions of inputs

Optimization becomes a necessity, not a choice

CORE PRINCIPLE

USE PROBLEM KNOWLEDGE TO LIMIT WORK

World Two algorithms:

- Make assumptions
- Use guarantees
- Trade generality for efficiency

WHAT COUNTS AS STRUCTURE

INFORMATION WE CAN RELY ON

Sorted order

Balanced hierarchy

Fixed constraints

Mathematical properties

Indexed access

WORLD ONE VS WORLD TWO

ALGORITHMIC MATURITY

Aspect	World One	World Two
Approach	Exhaustive	Selective
Assumptions	None	Explicit
Scalability	Poor	Controlled
Usage	Small problems	Real systems

SEARCH AS A CASE STUDY

SAME TASK, DIFFERENT THINKING

Problem: Find an element

Linear search → no assumptions

Binary search → ordered data

BINARY SEARCH CONCEPT

LOGARITHMIC REDUCTION

Check middle element

Eliminate half of remaining space

Repeat until resolved

WHY LOGARITHMIC TIME MATTERS

GROWTH RATE COMPARISON

Linear growth: proportional to input

Logarithmic growth: proportional to information

SORTING AS PREPROCESSING

INVEST TIME TO SAVE TIME

Used heavily in real systems

Sorting enables faster operations

Cost paid once

Benefit gained repeatedly

MERGE SORT

STRUCTURED DIVISION

Preferred in large-scale systems

Deterministic performance

Stable sorting

Predictable resource usage

QUICK SORT

PRACTICAL EFFICIENCY

Library implementations are heavily optimized variants

Average-case optimal

Cache-friendly

Dominates real-world usage

TREES AS ORGANIZATION

LOGARITHMIC ACCESS BY DESIGN

Binary Search Trees

Self-balancing trees

Hierarchical control of data

TREE USAGE IN PRACTICE

SYSTEM-LEVEL DEPENDENCE

File systems

Databases

Language parsers

Operating systems

HASHING

DIRECT ACCESS THROUGH DISTRIBUTION

Key → location mapping

Near-constant time access

Sacrifices order for speed

HASHMAPS IN INDUSTRY

DEFAULT DATA STRUCTURE

Caching

User sessions

Fast configuration lookup

ASSUMPTIONS (THE HIDDEN COST)

EFFICIENCY REQUIRES DISCIPLINE

Data must remain valid

Structure must be preserved

Violations degrade performance

FAILURE CASES

WHEN OPTIMIZATION BREAKS

Algorithms must be defended, not trusted blindly

Poor input distribution

Broken invariants

Worst-case exploitation

SECURITY CONNECTION

COMPLEXITY AS AN ATTACK VECTOR

Hash collision attacks

Algorithmic DoS

Worst-case exploitation

MAGIC SQUARE (OPTIMIZED)

FROM EXHAUSTIVE TO INTELLIGENT SEARCH

World TWO applied to World ONE

MAGIC SQUARE (OPTIMIZED)

FROM EXHAUSTIVE TO INTELLIGENT SEARCH

Fix center value

Enforce sum constraints early

Prune impossible branches

PRUNING

ELIMINATE FUTILE PATHS EARLY

Partial solutions evaluated

Immediate rejection if invalid

Exponential savings

DIVIDE AND CONQUER

STRUCTURED PROBLEM DECOMPOSITION

Solve smaller instances

Combine results

Foundation of many algorithms

SYSTEM DESIGN CONNECTION

ALGORITHMS SHAPE ARCHITECTURE

Indexing

Caching

Load balancing

Data pipelines

WHY ENGINEERS PREFER WORLD TWO

PREDICTABILITY

Known limits

Stable performance

Easier monitoring and scaling

COMMON STUDENT ERROR

OPTIMIZATION WITHOUT ASSUMPTIONS

Complex code

No real performance gain

Still brute force underneath

WORLD TWO THINKING SKILLS

QUESTIONS ENGINEERS ASK

What can I assume?

What can I precompute?

What can I discard early?

WHEN WORLD TWO IS NOT NEEDED

RIGHT TOOL, RIGHT PROBLEM

WHEN WORLD TWO IS NOT NEEDED

RIGHT TOOL, RIGHT PROBLEM

Small datasets

One-off computations

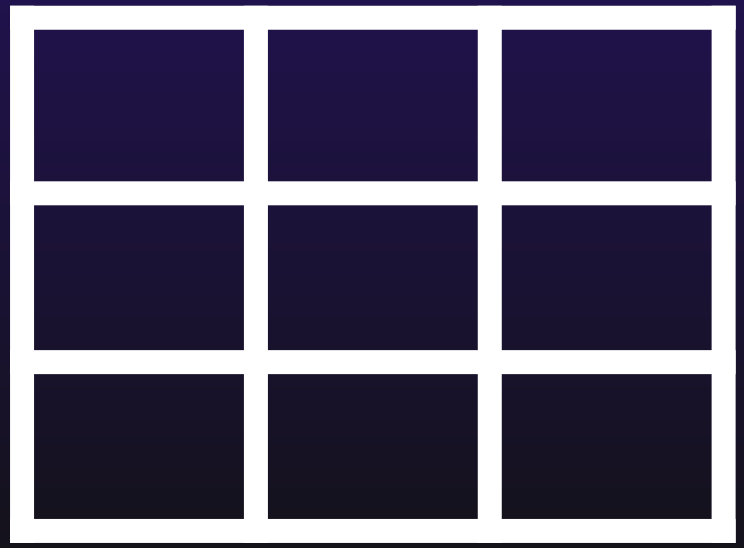
Rapid prototyping

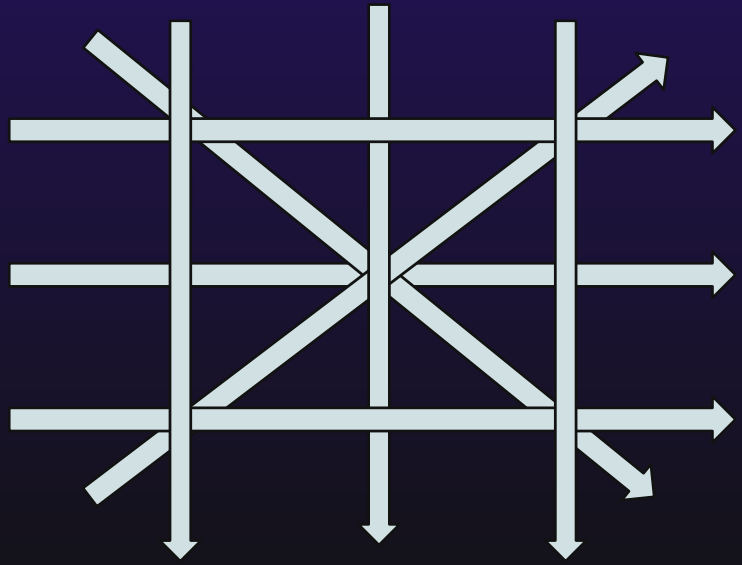
EXAMPLE - T000

TASK

INTEGERS: 1 2 3 4 5 6 7 8 9

15





INCREMENTAL CONSTRUCTION

BACKTRACKING (NAME)

Instead of filling the whole grid at once:

- Fill the grid cell by cell
-
- At each step, make a choice
-
- If the choice fails → undo it (backtrack)

This avoids generating all permutations.

CONSTRAINT CHECKING

STRUCTURE

The Magic Square has strict constraints:

- Numbers 1–9 used once
- Row sum = 15
- Column sum = 15
- Diagonal sum = 15

These constraints are checked:

- During construction
- Not at the end

PRUNING

EARLY TERMINATION

At each placement, we ask:

- Does any row sum exceed 15?
- Does any column sum exceed 15?
- Does any diagonal exceed 15?
- Is a number reused?

If any answer is yes THEN stop exploring this branch immediately

SEARCH SPACE REDUCTION

sdsds

Most partial grids are rejected early

The algorithm explores only valid paths

The total number of recursive calls drops dramatically

T006

THE CODE

https://github.com/AlaTooMirlanNurbekov/A006_Algorithmization_and_Programming/blob/main/0002_codes/T006.java

PROGRAM STARTS

WHAT ACTUALLY HAPPENS WHEN THE CODE RUNS

```java
public static void main(String[] args) {
    solve(pos: 0);
}
```

Java starts execution from main

solve(0) means → start filling the grid and start from position 0

At this moment the grid is completely empty (all zeros)

used[ ] is all false

PROGRAM STARTS

WHAT ACTUALLY HAPPENS WHEN THE CODE RUNS

```
public static void main(String[] args) {
    solve(pos: 0);
}
```

WHAT DOES pos : 0 MEAN?

PROGRAM STARTS

WHAT ACTUALLY HAPPENS WHEN THE CODE RUNS

```
int r = pos / 3;
int c = pos % 3;
```

We are filling grid[0][0]
(top-left corner)

For pos = 0:

r = 0 / 3 = 0

c = 0 % 3 = 0

TRYING NUMBERS (1 → 9)

WHAT ACTUALLY HAPPENS WHEN THE CODE RUNS

```
for (int num = 1; num <= 9; num++) {
```

Let me try every possible number in this cell, one by one

THE GRID

WHAT ACTUALLY HAPPENS WHEN THE CODE RUNS

1	0	0
0	0	0
0	0	0

SECOND CELL (0,1)

WHAT ACTUALLY HAPPENS WHEN THE CODE RUNS

Used numbers: 1

UNused numbers: 2 … 9

IMMEDIATE VALIDATION (KEY WORLD TWO IDEA)

WHAT ACTUALLY HAPPENS WHEN THE CODE RUNS

```
if (isValid(r, c)) {
```

This is THE CORE DIFFERENCE from brute force

Instead of waiting until the end, we ask:

“Did placing 1 already break something?”

WHAT isValid(0,0) CHECKS

WHAT ACTUALLY HAPPENS WHEN THE CODE RUNS

Row 0:

Sum = 1

Not complete → OK

Column 0:

Sum = 1

Not complete → OK

Main diagonal:

(0,0) is on diagonal

Sum = 1

Not complete → OK

Nothing is broken

isValid returns **true**

MOVE FORWARD (RECURSION)

WHAT ACTUALLY HAPPENS WHEN THE CODE RUNS

```
solve(pos + 1);
```

Good choice so far. Let’s fill the next cell

POS = 1 (SECOND CELL)

WHAT ACTUALLY HAPPENS WHEN THE CODE RUNS

For pos = 1:

r = 0

c = 1

We are filling grid[0][1]

The algorithm again tries numbers from 1 → 9.

num = 1

used[1] == true → SKIPPED

num = 2

STATUS: allowed

THE GRID

WHAT ACTUALLY HAPPENS WHEN THE CODE RUNS

1	2	0
0	0	0
0	0	0

POS = 1 (SECOND CELL)

WHAT ACTUALLY HAPPENS WHEN THE CODE RUNS

isValid(0,1):

Row sum = 3 → OK

Column sum = 2 → OK

No diagonal involved

Valid → go deeper

THIS CONTINUES CELL BY CELL

WHAT ACTUALLY HAPPENS WHEN THE CODE RUNS

The algorithm keeps repeating the same pattern:

1. Pick next empty cell
2. Try numbers 1 → 9
3. Skip used numbers
4. Place number
5. Immediately check constraints
6. Continue or go back

PRUNING IN ACTION

WHAT ACTUALLY HAPPENS WHEN THE CODE RUNS

At some point, the algorithm may try something like:

Row becomes: 8 + 7 + 4 = 19

Inside isValid --> if (sum > 15) return false;

This branch is killed immediately

Number is removed

used[num] = false

Try next number

Entire future subtree is skipped

This is PRUNING

BACKTRACKING

WHAT ACTUALLY HAPPENS WHEN THE CODE RUNS

```
grid[r][c] = 0;
used[num] = false;
```

Undo the move. Pretend this never happened

Then the algorithm tries the next number or goes back to the previous cell

REACHING A FULL SOLUTION

WHAT ACTUALLY HAPPENS WHEN THE CODE RUNS

```
grid[r][c] = 0;
used[num] = false;
```

Eventually we get to point when → if (pos == 9)

Means all 9 cells are filled and all checks passed

SUMMARY

WHAT ACTUALLY HAPPENS WHEN THE CODE RUNS

The grid was built one cell at a time

After every placement, rules were checked

Bad paths were cut early

Only promising paths were explored

The solution was inevitable, not lucky

NEXT WEEK PREVIEW

COMING SOON

World THREE

FROM STRUCTURE TO STRATEGY